

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

Facultad de Ciencias de la Computación y Diseño Digital

Ingeniería de Software

INFORME DE PRÁCTICA

Desarrollo de una aplicación web MVC con ASP.NET Core 10: CRUD de Producto con validación y protección CSRF

Asignatura: Aplicaciones Web (5.º semestre) — Unidad II

Docente: Dr. Gleiston Cicerón Guerrero Ulloa, Ph.D.

Período: PPA 2026-2027

Estudiante: Figueroa Morales Bryan Jaier

Repositorio: https://github.com/011011bjf/Practica_Experimental_ASP.NET.git

Fecha: 20 de junio de 2026

Tabla de contenido

Tabla de contenido	2
1. Introducción	3
2. Objetivos	3
2.1 Objetivo general	3
2.2 Objetivos específicos.....	3
3. Marco teórico	3
3.1 El patrón MVC en ASP.NET Core.....	3
3.2 Validación de datos con Data Annotations.....	4
3.3 Protección CSRF con AntiForgeryToken	4
4. Herramientas y entorno de desarrollo	4
5. Desarrollo de la práctica	4
5.1 Creación del proyecto base	4
5.2 Estructura del proyecto.....	5
5.3 Modelo Producto	5
5.4 Controlador ProductoController	6
5.5 Vistas Razor	7
5.6 Validación cliente y servidor	7
Procedimiento de verificación	7
5.7 Control de versiones.....	8
6. Pruebas y verificación	8
7. Resultados obtenidos.....	8
8. Conclusiones	9
9. Recomendaciones y trabajo futuro.....	9

1. Introducción

Este informe documenta el desarrollo de una práctica del curso Aplicaciones Web, correspondiente a la Unidad II de la guía “Desarrollo de Aplicaciones Web con ASP.NET Core 10”. A diferencia de los ejemplos de la guía centrados en Minimal APIs, esta práctica explora el patrón Model-View-Controller (MVC) clásico, descrito en la sección 11 de dicha guía, aplicado a un caso concreto: la gestión de un catálogo de productos.

El proyecto resultante, denominado MiPrimerMVC, implementa un flujo completo de alta, listado y baja de productos, con énfasis en dos aspectos que suelen pasarse por alto en ejercicios introductorios: la validación de datos en dos capas (cliente y servidor) y la protección contra ataques de tipo CSRF (Cross-Site Request Forgery) mediante el token anti-falsificación que ofrece ASP.NET Core de forma nativa.

2. Objetivos

2.1 Objetivo general

Desarrollar una aplicación web siguiendo el patrón MVC en ASP.NET Core 10, que permita gestionar un catálogo de productos de forma segura, aplicando validación de datos y protección contra falsificación de solicitudes entre sitios (CSRF).

2.2 Objetivos específicos

- Crear la estructura base de un proyecto ASP.NET Core MVC equivalente a la generada por `dotnet new mvc`.
- Definir un modelo de dominio (Producto) validado con Data Annotations.
- Implementar un controlador (ProductoController) con las acciones Index, Create y Delete.
- Construir vistas Razor que muestren los datos en una tabla HTML y formularios protegidos con `AntiForgeryToken`.
- Verificar que la validación ocurre tanto en el cliente (JavaScript) como en el servidor (ModelState).
- Versionar el trabajo en una rama de funcionalidad (feature/practica-aspnet) siguiendo un flujo de Git estándar.
- Documentar el proyecto en un README con instrucciones de ejecución.

3. Marco teórico

3.1 El patrón MVC en ASP.NET Core

En el patrón Model-View-Controller, un controlador recibe la petición HTTP, interactúa con el modelo de datos y selecciona una vista Razor (.cshtml) que se encarga de renderizar el HTML

de respuesta. Este modelo se habilita en Program.cs mediante `AddControllersWithViews()` y se enruta con `MapControllerRoute`, usando por defecto el patrón `{controller=Home}/{action=Index}/{id?}`. Es el estilo recomendado cuando el HTML se genera en el servidor, sin un frontend SPA independiente.

3.2 Validación de datos con Data Annotations

Las Data Annotations son atributos que se aplican sobre las propiedades de un modelo (por ejemplo, `[Required]`, `[StringLength]` o `[Range]`) y que ASP.NET Core interpreta en dos momentos distintos: al construir el formulario, generando atributos HTML5 y metadatos para `jquery-validation-unobtrusive` (validación de cliente); y al recibir el POST, poblando la propiedad `ModelState` del controlador (validación de servidor). Definir las reglas una sola vez, en el modelo, evita duplicarlas y mantiene ambas capas sincronizadas.

3.3 Protección CSRF con AntiForgeryToken

Un ataque CSRF ocurre cuando un sitio malicioso induce al navegador de una víctima autenticada a enviar una petición no deseada hacia otra aplicación (por ejemplo, un borrado) aprovechando que el navegador adjunta automáticamente las cookies de sesión. ASP.NET Core mitiga esto con un token anti-falsificación: un valor aleatorio, único por usuario, que se incrusta en el formulario con `@Html.AntiForgeryToken()` (o automáticamente por el `FormTagHelper`) y se exige en el servidor con el atributo `[ValidateAntiForgeryToken]`. Si el token no llega o no coincide, la petición se rechaza con 400 Bad Request antes de ejecutar la lógica del controlador.

4. Herramientas y entorno de desarrollo

- SDK de .NET 10 (LTS)
- Lenguaje C# 14
- ASP.NET Core 10 — MVC con vistas Razor
- Visual Studio Code con la extensión C# Dev Kit (IDE de referencia de la guía del curso)
- Bootstrap 5 y jQuery (cargados vía CDN para la interfaz)
- `jquery-validation` / `jquery-validation-unobtrusive` (validación de cliente)
- Git para control de versiones

5. Desarrollo de la práctica

5.1 Creación del proyecto base

El punto de partida equivalente al comando `dotnet new mvc -n MiPrimerMVC` es un proyecto con archivo de proyecto (.csproj) orientado al SDK `Microsoft.NET.Sdk.Web`, apuntando a `net10.0`, junto con un Program.cs que registra los servicios MVC y configura el pipeline de middleware.

Código 1. Program.cs — registro de servicios y pipeline de middleware.

```
var builder = WebApplication.CreateBuilder(args);
```

```

builder.Services.AddControllersWithViews();

var app = builder.Build();

if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Home/Error");
    app.UseHsts();
}

app.UseHttpsRedirection();
app.UseStaticFiles();
app.UseRouting();
app.UseAuthorization();

app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");

app.Run();

```

5.2 Estructura del proyecto

Siguiendo la organización por carpetas recomendada en la guía del curso (sección 12.1), el proyecto separa modelos, controladores y vistas:

```

MiPrimerMVC/
├── Controllers/
│   ├── HomeController.cs
│   └── ProductoController.cs    (Index, Create, Delete)
├── Models/
│   ├── Producto.cs            (Data Annotations)
│   └── ErrorViewModel.cs
├── Views/
│   ├── Producto/
│   │   ├── Index.cshtml       (tabla HTML)
│   │   ├── Create.cshtml      (formulario + AntiForgeryToken)
│   │   └── Delete.cshtml      (confirmación POST + CSRF)
│   └── Shared/_Layout.cshtml
├── wwwroot/
├── Program.cs
└── MiPrimerMVC.csproj

```

5.3 Modelo Producto

El modelo concentra todas las reglas de negocio relativas a la forma de los datos. Cada Data Annotation define tanto un mensaje de error como el criterio de validación, reutilizado automáticamente en cliente y servidor.

Código 2. Models/Producto.cs (extracto).

```

public class Producto
{
    public int Id { get; set; }

    [Required(ErrorMessage = "El nombre es obligatorio.")]
    [StringLength(100, MinimumLength = 2)]
    public string Nombre { get; set; } = string.Empty;

    [Required, StringLength(500)]
    public string Descripcion { get; set; } = string.Empty;

    [Required, Range(0.01, 1_000_000)]
    public decimal Precio { get; set; }

    [Required, Range(0, int.MaxValue)]
    public int Stock { get; set; }

    public DateTime FechaRegistro { get; set; } = DateTime.UtcNow;
}

```

5.4 Controlador ProductoController

El controlador expone tres acciones. Index lista los productos; Create muestra el formulario (GET) y procesa el alta (POST); Delete muestra una página de confirmación (GET) y ejecuta el borrado real en una acción separada (POST), siguiendo la convención Post-Redirect-Get y evitando que un simple enlace pueda disparar un borrado.

Código 3. Controllers/ProductoController.cs (extracto — acción Create).

```

[HttpPost]
[ValidateAntiForgeryToken]
public IActionResult Create(
    [Bind("Nombre,Descripcion,Precio,Stock")] Producto producto)
{
    if (!ModelState.IsValid)
    {
        return View(producto); // se reenvían los errores a la vista
    }

    producto.Id = _siguienteId++;
    producto.FechaRegistro = DateTime.UtcNow;
    _productos.Add(producto);

    return RedirectToAction(nameof(Index));
}

```

Código 4. Controllers/ProductoController.cs (extracto — borrado en dos pasos).

```

// GET: solo muestra la confirmación, no modifica datos
public IActionResult Delete(int? id) { /* ... */ }

```

```
// POST: requiere token CSRF válido para ejecutarse
[HttpPost, ActionName("Delete")]
[ValidateAntiForgeryToken]
public IActionResult DeleteConfirmed(int id)
{
    var producto = _productos.FirstOrDefault(p => p.Id == id);
    if (producto is null) return NotFound();

    _productos.Remove(producto);
    return RedirectToAction(nameof(Index));
}
```

5.5 Vistas Razor

La vista Index.cshtml recorre la colección de productos y construye una tabla HTML con Bootstrap. Las vistas Create.cshtml y Delete.cshtml incluyen explícitamente el token anti-falsificación mediante `@Html.AntiForgeryToken()`, de modo que el campo oculto `__RequestVerificationToken` viaje junto con cada envío POST.

Código 5. Views/Producto/Delete.cshtml (extracto).

```
<form asp-action="Delete" method="post">
    @Html.AntiForgeryToken()
    <input type="hidden" asp-for="Id" />
    <button type="submit" class="btn btn-danger">
        Eliminar definitivamente
    </button>
</form>
```

5.6 Validación cliente y servidor

En el navegador, la librería jquery-validation-unobtrusive lee los atributos `data-val-*` generados a partir de las Data Annotations e impide el envío del formulario mientras existan errores, mostrando los mensajes en los elementos `<span asp-validation-for="...">`. En el servidor, el mismo conjunto de reglas se evalúa de nuevo sobre ModelState dentro de la acción Create, de modo que una petición enviada sin pasar por el formulario (por ejemplo, con curl o con JavaScript deshabilitado) sigue quedando protegida.

Procedimiento de verificación

1. Enviar el formulario de Create con el campo Nombre vacío: el envío se bloquea en el cliente y aparece el mensaje de error junto al campo, sin recargar la página.
2. Deshabilitar JavaScript en el navegador (o usar una petición HTTP directa) y reenviar el mismo formulario: el servidor responde con la vista Create, ahora con los errores generados por ModelState.IsValid.
3. Intentar un POST a /Producto/Create o /Producto/Delete sin el campo `__RequestVerificationToken`: ASP.NET Core responde 400 Bad Request antes de ejecutar la lógica del controlador, confirmando que [ValidateAntiForgeryToken] está activo.

5.7 Control de versiones

El trabajo se aisló en una rama de funcionalidad dedicada, siguiendo la convención `feature/<nombre-descriptivo>`, y se registró en un único commit que agrupa el alcance completo de la práctica:

```
git init
git checkout -b feature/practica-aspnet
git add .
git commit -m "Practica ASP.NET Core MVC: CRUD parcial de Producto
               con CSRF y validacion server-side"

# Publicación en el repositorio remoto del estudiante:
git remote add origin <URL_DEL_REPOSITORIO>
git push -u origin feature/practica-aspnet
```

6. Pruebas y verificación

Dado que el entorno de generación de este informe no contó con el SDK de .NET ni acceso al registro de paquetes NuGet, las pruebas de compilación y ejecución (`dotnet build`, `dotnet run`) deben completarse en el equipo del estudiante, siguiendo el README del proyecto. No obstante, se aplicaron las siguientes verificaciones estáticas durante el desarrollo:

- Revisión de balance de llaves y estructura sintáctica en los archivos `.cs`.
- Revisión cruzada de cada acción del controlador contra su vista correspondiente (nombres de campos, `asp-for`, rutas de acción).
- Verificación de que toda acción que modifica estado (Create POST, Delete POST) está decorada con `[HttpPost]` y `[ValidateAntiForgeryToken]`.
- Verificación de que el borrado no es alcanzable mediante una petición GET.

Se recomienda que, antes de la entrega final, el estudiante ejecute `dotnet build`, `dotnet run` y repita manualmente el procedimiento de verificación descrito en la sección 5.6, idealmente documentando los resultados con capturas de pantalla del navegador.

7. Resultados obtenidos

La tabla siguiente resume el cumplimiento de cada punto solicitado en el enunciado de la práctica frente a su evidencia concreta dentro del proyecto.

#	Requisito solicitado	Estado	Evidencia / archivo
1	Crear el proyecto base (<code>dotnet new mvc -n MiPrimerMVC</code>)	Completado	Estructura del proyecto y <code>MiPrimerMVC.csproj</code>
2	ProductoController con acciones Index y Create	Completado	<code>Controllers/ProductoController.cs</code>
3	Modelo Producto con Data Annotations	Completado	<code>Models/Producto.cs</code>
4	Vista Razor con <code>ValidateAntiForgeryToken</code>	Completado	<code>Views/Producto/Create.cshtml</code> y <code>Delete.cshtml</code>

5	Verificar validación server-side	Completado	ModelState.IsValid en ProductoController.Create
6	Tabla HTML con listado de productos	Completado	Views/Producto/Index.cshtml
7	Acción Delete con confirmación POST y CSRF	Completado	Delete (GET) + DeleteConfirmed (POST)
8	Validación cliente y servidor	Completado	jquery-validation-unobtrusive + ModelState
9	Subir a rama feature/practica-aspnet	Completado en local*	git checkout -b feature/practica-aspnet + commit
10	README con instrucciones de ejecución	Completado	README.md

8. Conclusiones

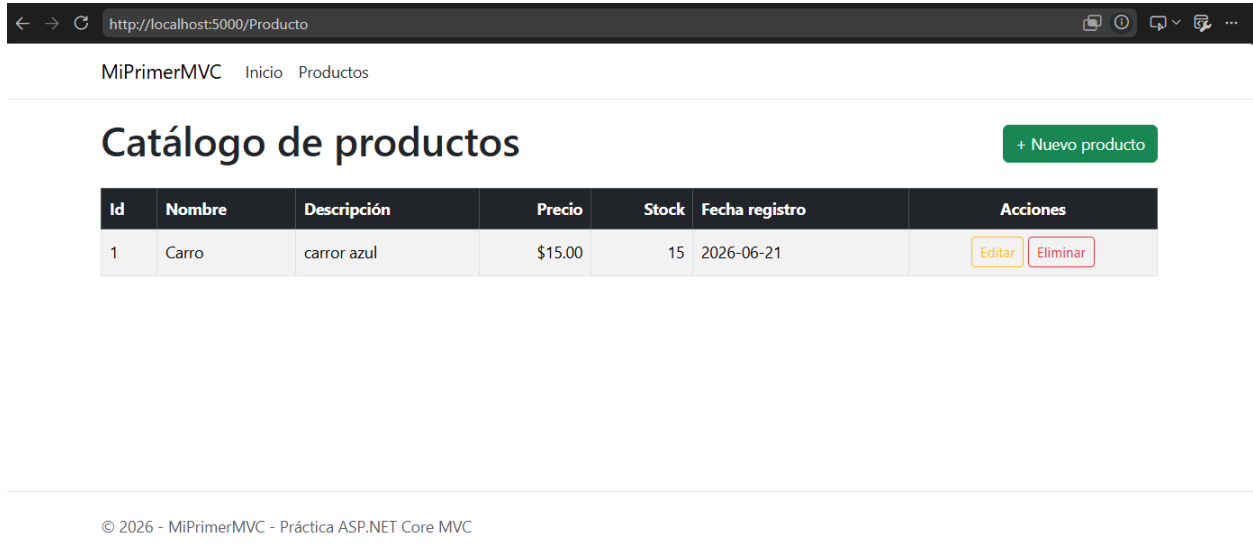
- El patrón MVC de ASP.NET Core permite separar claramente la lógica de presentación (vistas Razor), el flujo de control (controlador) y las reglas de datos (modelo), lo que facilita razonar sobre cada capa de forma independiente.
- Definir las reglas de validación una sola vez, mediante Data Annotations en el modelo, evita duplicación de lógica entre cliente y servidor y reduce el riesgo de que ambas capas queden desincronizadas.
- El token anti-falsificación de ASP.NET Core es sencillo de incorporar (@Html.AntiForgeryToken() + [ValidateAntiForgeryToken]) y constituye una defensa efectiva contra CSRF sin requerir librerías externas.
- Separar la confirmación de borrado (GET) de la ejecución real (POST) evita que acciones destructivas se disparen por accidente mediante enlaces, prefetch del navegador o rastreadores automáticos.

9. Recomendaciones y trabajo futuro

- Sustituir el repositorio en memoria por un DbContext de Entity Framework Core con PostgreSQL, siguiendo la sección 7 de la guía del curso, para persistir los datos entre ejecuciones.
- Añadir la acción Edit (GET/POST) siguiendo el mismo patrón aplicado a Create, para completar el CRUD.
- Incorporar paginación en Index cuando el catálogo crezca, reutilizando el patrón Skip/Take mostrado en la guía.
- Proteger las acciones de escritura con [Authorize] una vez integrada autenticación (JWT o ASP.NET Core Identity).
- Agregar pruebas automatizadas (por ejemplo, con WebApplicationFactory) que cubran tanto el camino feliz como los casos de validación fallida y token CSRF ausente.

10. Anexos

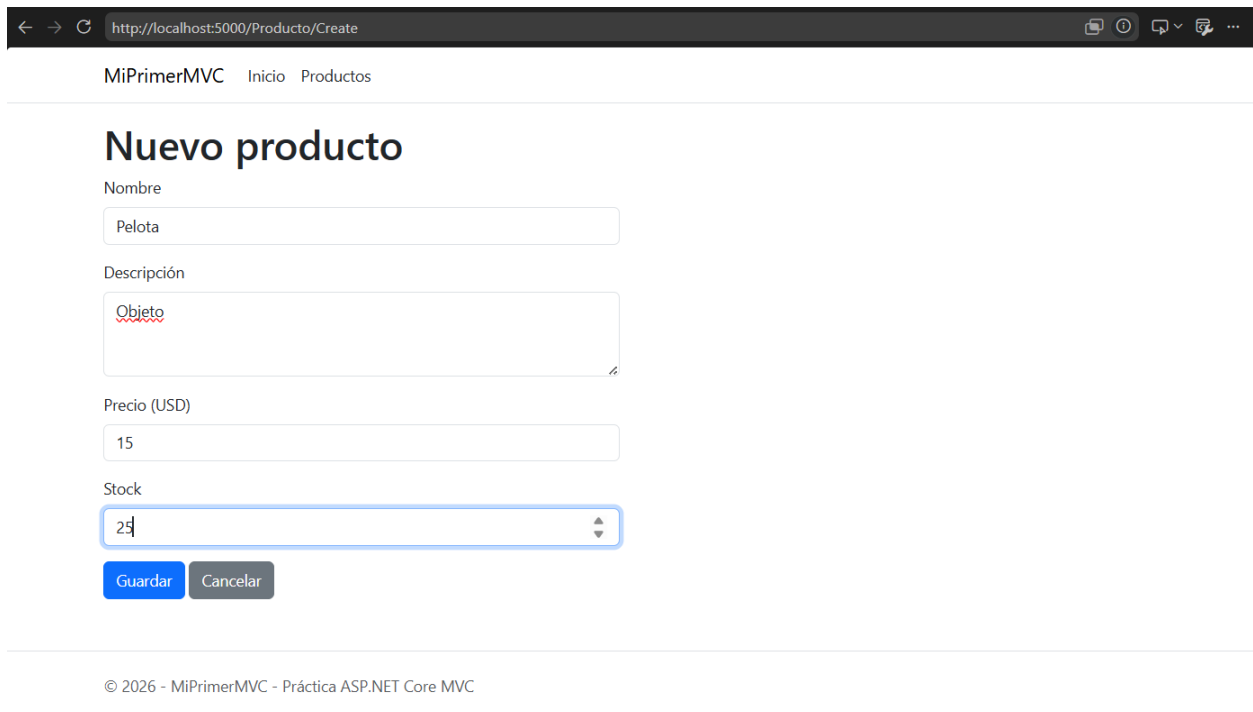
Pruebas de ejecución



The screenshot shows a web browser at the URL `http://localhost:5000/Producto`. The page title is 'Catálogo de productos'. At the top right, there is a green button labeled '+ Nuevo producto'. Below the title is a table with the following data:

Id	Nombre	Descripción	Precio	Stock	Fecha registro	Acciones
1	Carro	carror azul	\$15.00	15	2026-06-21	Editar Eliminar

At the bottom of the page, there is a footer: '© 2026 - MiPrimerMVC - Práctica ASP.NET Core MVC'.



The screenshot shows a web browser at the URL `http://localhost:5000/Producto/Create`. The page title is 'Nuevo producto'. It contains a form with the following fields:

- Nombre:
- Descripción:
- Precio (USD):
- Stock:

At the bottom of the form are two buttons: 'Guardar' (blue) and 'Cancelar' (gray). At the bottom of the page, there is a footer: '© 2026 - MiPrimerMVC - Práctica ASP.NET Core MVC'.

← → ↻ http://localhost:5000/Producto

MiPrimerMVC Inicio Productos

Producto "Pelota" creado correctamente. ✕

Catálogo de productos

+ Nuevo producto

Id	Nombre	Descripción	Precio	Stock	Fecha registro	Acciones
1	Carro	carror azul	\$15.00	15	2026-06-21	Editar Eliminar
2	Pelota	Objeto	\$15.00	25	2026-06-21	Editar Eliminar

© 2026 - MiPrimerMVC - Práctica ASP.NET Core MVC

Ejecución de la prueba en clase

En la carpeta realizado en clases esta este proyecto

The screenshot displays a development environment with three main components:

- pgAdmin 4:** A window showing the PostgreSQL database structure. The 'estudiantes_db' database is selected, and the 'estudiantes' table is visible. The table has columns: Id (PK, integer), Nombre (text), Correo (text), Carrera (text), and Creado (timestamp with time zone). The data shows two rows: (1, 'bryan', 'hola@hola', 'Software', '2026-06-20 19:19') and (2, 'Juan Pérez', 'juan.perez@ejemplo.com', 'Ingeniería en Sistemas', '2026-06-20 19:19').
- REST Client:** A window showing a POST request to 'http://localhost:5232/v1/estudiantes'. The request body is a JSON object:

```
{ "nombre": "bryan", "correo": "hola@hola", "carrera": "Software" }
```

. The response is a 200 status code.
- Code Editor:** A window showing the 'Program.cs' file in the 'ASPNET' project. The file contains the following code:


```
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Threading.Tasks;

namespace ASPNET
{
    public class Program
    {
        public static void Main(string[] args)
        {
            CreateWebHostBuilder(args).Build().Run();
        }

        public static IWebHostBuilder CreateWebHostBuilder(string[] args) =>
            WebHostBuilder.CreateDefaultBuilder(args)
                .UseStartup<Startup>()
                .Build();
    }
}
```